PharmaKeep

Micro-Pharmacy Inventory & Expiry Tracking System

Milestones 2 – 5 | Database Systems Lab

Student	Awais Qarni
ID / Major	BSSE | Group B
Project	PharmaKeep
Submission	17 May 2026
GitHub	github.com/awaisqarni/pharmaKeep

MILESTONE 2 — Normalization (1NF → 2NF → 3NF)

2.1 What is Database Normalization?

Normalization is the process of structuring a relational database schema to eliminate data redundancy and prevent update, insertion, and deletion anomalies. The PharmaKeep schema was designed with normalization in mind from the outset. The walkthrough below formally proves that every table satisfies Third Normal Form (3NF).

2.2 First Normal Form (1NF)

Rule: Every column must contain only atomic (indivisible) values; each row must be unique; no repeating groups.

Table	Primary Key	Atomicity Justification	1NF
Staff	StaffID (PK, AI)	All columns atomic; no multi-valued fields	✓ Pass
Category	CategoryID (PK, AI)	Single attribute; no groups	✓ Pass
Medicines	MedicineID (PK, AI)	Each attribute indivisible; one row per product	✓ Pass
Suppliers	SupplierID (PK, AI)	ContactPerson is one person per supplier row	✓ Pass
Batches	BatchID (PK, AI)	Each batch is one row; prices not embedded as lists	✓ Pass
Sales	SaleID (PK, AI)	Line-items moved to SaleItems — no repeating groups	✓ Pass
SaleItems	SaleItemID (PK, AI)	One product line per row; atomic quantities/prices	✓ Pass

2.3 Second Normal Form (2NF)

Rule: Must be in 1NF, and every non-key attribute must depend on the *entire* primary key (no partial dependencies). Partial dependencies only arise when a table has a composite PK.

Every table in PharmaKeep uses a **single-column surrogate primary key** (AUTO_INCREMENT INT). A partial dependency requires at least two columns in the PK — therefore partial dependencies are structurally impossible in this schema. The one composite *unique* constraint (BatchNumber + MedicineID in Batches) is a uniqueness constraint, not the PK, so it does not introduce partial dependencies.

Table	Primary Key Type	2NF Justification	2NF
Staff	Single PK (StaffID)	No composite PK → no partial deps	✓ Pass
Category	Single PK (CategoryID)	No composite PK → no partial deps	✓ Pass
Medicines	Single PK (MedicineID)	CategoryID is a FK reference, not part of PK	✓ Pass
Suppliers	Single PK (SupplierID)	No composite PK → no partial deps	✓ Pass
Batches	Single PK (BatchID)	MedicineID, SupplierID are FKs, not PK parts	✓ Pass
Sales	Single PK (SaleID)	StaffID is a FK; no composite PK	✓ Pass
SaleItems	Single PK (SaleItemID)	SaleID, BatchID are FKs; PK is single column	✓ Pass

2.4 Third Normal Form (3NF)

Rule: Must be in 2NF, and no non-key attribute may transitively depend on the primary key through another non-key attribute.

Table	Potential Transitive Dep.	3NF Resolution	3NF

Staff	Role → permissions	Role is ENUM; permission logic is application-level, not stored as a separate table. No transitive dep.	✓ Pass
Category	None identified	Only one non-key attribute (CategoryName). Nothing to be transitive through.	✓ Pass
Medicines	CategoryID → CategoryName	CategoryName lives in Category table; Medicines stores only the FK. Dependency eliminated by decomposition.	✓ Pass
Suppliers	None identified	All attributes (Name, Contact, Phone, Email) directly describe the supplier row.	✓ Pass
Batches	MedicineID → BrandName, SupplierID → SupplierName	Medicine & Supplier details stored in their own tables; Batches holds only FKs. No transitive deps.	✓ Pass
Sales	StaffID → FullName	Staff details in Staff table; Sales holds only StaffID FK. Fully 3NF.	✓ Pass
SaleItems	BatchID → ExpiryDate, MedicineID	All Batch attributes reside in Batches table. SaleItems holds BatchID FK and UnitPrice snapshot.	✓ Pass

Conclusion: The PharmaKeep schema satisfies 1NF, 2NF, and 3NF across all seven tables. The design correctly separates concerns — medicine catalogue, batch inventory, supplier registry, staff management, and sales transactions — each in its own normalized entity.

MILESTONE 3 — Dataset Preparation & Dataflow Description

3.1 Data Strategy

Because PharmaKeep is an academic project without access to real patient or transaction records, **structured synthetic data** was generated programmatically using Python. The data is deterministic (seeded with random.seed(42)) so results are fully reproducible. Realistic medicine names, supplier contacts, batch numbers, and price ranges were hand-crafted to reflect a typical Pakistani retail pharmacy.

3.2 Dataset Summary

Table	Row Count	Notes
Staff	5	2 Admins, 3 Pharmacists — covers all ENUM values
Category	8	Antibiotics, Analgesics, Vitamins, Antihypertensives, Antidiabetics, Antihistamines, Gastrointestinals, Dermatologicals
Medicines	15	Real brand + generic name pairs (Panadol/Paracetamol, Augmentin/Amoxicillin, etc.)
Suppliers	5	Fictional Pakistani distributors with realistic contact data
Batches	51	2–4 batches per medicine; mix of valid and expired records to test expiry queries
Sales	100	Spread across Jan 2025 – Apr 2026; processed by Pharmacist staff only
SaleItems	247	Only valid (non-expired, in-stock) batches; SubTotal = Qty × UnitPrice

3.3 Dataflow Description

The diagram below shows how data enters and moves through the PharmaKeep system during normal pharmacy operations.

1. Stock Arrival

A Supplier delivers medicines. A Staff member (Admin) records a new Batch — linking it to an existing Medicine and the Supplier. BatchNumber + ExpiryDate + QuantityReceived are captured. QuantityOnHand is initialised = QuantityReceived.

2. Medicine Lookup

At the point-of-sale, the Pharmacist searches the Medicines catalogue by BrandName or GenericName. The system filters Batches WHERE ExpiryDate > TODAY AND QuantityOnHand > 0, presenting only sellable stock.

3. Sale Transaction

A new Sales record is created, stamped with SaleDateTime, linked to the Pharmacist's StaffID, and assigned a PaymentMethod. For each item the customer buys, a SaleItems row is inserted — referencing the specific BatchID to ensure correct inventory deduction.

4. Inventory Deduction

On commit, a trigger (or application logic) decrements Batches.QuantityOnHand -= SaleItems.QuantitySold for every line item. This ensures real-time stock accuracy at the batch level.

5. Reporting

Management queries join Sales ↔ SaleItems ↔ Batches ↔ Medicines ↔ Suppliers to produce revenue reports, expiry alerts, and staff performance dashboards.

3.4 CSV File Index

File	Rows	Columns
Staff.csv	5	StaffID, FullName, Username, PasswordHash, Role
Category.csv	8	CategoryID, CategoryName
Medicines.csv	15	MedicineID, BrandName, GenericName, CategoryID, Description
Suppliers.csv	5	SupplierID, SupplierName, ContactPerson, PhoneNumber, Email
Batches.csv	51	BatchID, BatchNumber, MedicineID, SupplierID, ExpiryDate, QuantityReceived, QuantityOnHand, CostPrice, SellingPrice
Sales.csv	100	SaleID, SaleDateTime, StaffID, TotalAmount, PaymentMethod
SaleItems.csv	247	SaleItemID, SaleID, BatchID, QuantitySold, UnitPrice, SubTotal

MILESTONE 4 — CREATE TABLE Statements (DDL)

4.1 Overview

All tables are created in the pharmaKeep database using **MySQL 8.x** with the InnoDB storage engine (supports transactions and FK enforcement). The full script is committed to GitHub as sql/M4_DDL_PharmaKeep.sql. Each constraint is explicitly named for easy debugging.

4.2 Constraint Summary

Table	PRIMARY KEY	UNIQUE	FOREIGN KEYS	CHECK
Staff	pk_Staff	uq_Staff_User	—	—
Category	pk_Category	uq_CategoryName	—	—
Medicines	pk_Medicines	—	fk_Medicines_Cat → Category	—
Suppliers	pk_Suppliers	uq_SupplierName	—	—
Batches	pk_Batches	uq_BatchNumMed (composite)	fk_Batches_Med, fk_Batches_Sup	chk_QtyRcvd, chk_QtyOnHand, chk_CostPos, chk_SellPos
Sales	pk_Sales	—	fk_Sales_Staff → Staff	chk_TotalAmt
SaleItems	pk_SaleItems	—	fk_SaleItems_Sale, fk_SaleItems_Batch	chk_QtySold, chk_UnitPrice, chk_SubTotal

4.3 DDL Script (key excerpts)

Table: Staff

```
CREATE TABLE Staff (
  StaffID      INT           NOT NULL AUTO_INCREMENT,
  FullName     VARCHAR(100)  NOT NULL,
  Username     VARCHAR(50)   NOT NULL,
  PasswordHash VARCHAR(255)  NOT NULL,
  Role         ENUM('Admin','Pharmacist') NOT NULL,
  CONSTRAINT pk_Staff      PRIMARY KEY (StaffID),
  CONSTRAINT uq_Staff_User UNIQUE (Username)
) ENGINE=InnoDB;
```

Table: Batches (core inventory)

```
CREATE TABLE Batches (
  BatchID          INT           NOT NULL AUTO_INCREMENT,
  BatchNumber      VARCHAR(50)   NOT NULL,
  MedicineID       INT           NOT NULL,
  SupplierID       INT           NOT NULL,
  ExpiryDate       DATE          NOT NULL,
  QuantityReceived INT           NOT NULL,
  QuantityOnHand   INT           NOT NULL,
  CostPrice        DECIMAL(10,2) NOT NULL,
  SellingPrice     DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_Batches     PRIMARY KEY (BatchID),
  CONSTRAINT uq_BatchNumMed UNIQUE (BatchNumber, MedicineID),
  CONSTRAINT fk_Batches_Med FOREIGN KEY (MedicineID)
      REFERENCES Medicines(MedicineID) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT fk_Batches_Sup FOREIGN KEY (SupplierID)
      REFERENCES Suppliers(SupplierID) ON UPDATE CASCADE ON DELETE RESTRICT,
```

```
  CONSTRAINT chk_QtyRcvd    CHECK (QuantityReceived > 0),
  CONSTRAINT chk_QtyOnHand  CHECK (QuantityOnHand  >= 0)
) ENGINE=InnoDB;
```

Table: SaleItems (bridge table)

```
CREATE TABLE SaleItems (
  SaleItemID   INT           NOT NULL AUTO_INCREMENT,
  SaleID       INT           NOT NULL,
  BatchID      INT           NOT NULL,
  QuantitySold INT           NOT NULL,
  UnitPrice    DECIMAL(10,2) NOT NULL,
  SubTotal     DECIMAL(10,2) NOT NULL,
  CONSTRAINT pk_SaleItems       PRIMARY KEY (SaleItemID),
  CONSTRAINT fk_SaleItems_Sale  FOREIGN KEY (SaleID)
      REFERENCES Sales(SaleID)   ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_SaleItems_Batch FOREIGN KEY (BatchID)
      REFERENCES Batches(BatchID) ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT chk_QtySold  CHECK (QuantitySold > 0),
  CONSTRAINT chk_SubTotal CHECK (SubTotal     > 0)
) ENGINE=InnoDB;
```

MILESTONE 5 — Database Population & Validation Queries

5.1 Loading Strategy

Data is loaded using MySQL's **LOAD DATA INFILE** command — the fastest bulk-load method for CSV files. Foreign key checks are temporarily disabled during loading (SET FOREIGN_KEY_CHECKS = 0) and re-enabled immediately after (SET FOREIGN_KEY_CHECKS = 1). This avoids ordering constraints while still enforcing integrity post-load via validation queries.

Load order (respects FK hierarchy): Staff → Category → Medicines → Suppliers → Batches → Sales → SaleItems

5.2 Load Command Template

```
LOAD DATA INFILE '/var/lib/mysql-files/Staff.csv'
INTO TABLE Staff
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES  TERMINATED BY '\n'
IGNORE 1 ROWS
(StaffID, FullName, Username, PasswordHash, Role);
```

5.3 Validation Query Suite

V1 — Row Counts

Confirms every table loaded the expected number of rows.

```
SELECT 'Staff'     AS TableName, COUNT(*) AS RowCount FROM Staff
UNION ALL SELECT 'Category',   COUNT(*) FROM Category
UNION ALL SELECT 'Medicines',  COUNT(*) FROM Medicines
UNION ALL SELECT 'Suppliers',  COUNT(*) FROM Suppliers
UNION ALL SELECT 'Batches',    COUNT(*) FROM Batches
UNION ALL SELECT 'Sales',      COUNT(*) FROM Sales
UNION ALL SELECT 'SaleItems',  COUNT(*) FROM SaleItems;
```

V2 — NULL Checks

Zero violations expected on all NOT NULL columns.

```
SELECT 'Staff: NULL violations' AS Check_Name, COUNT(*) AS Violations
FROM Staff WHERE FullName IS NULL OR Username IS NULL OR Role IS NULL
UNION ALL
SELECT 'Batches: NULL violations', COUNT(*)
FROM Batches WHERE ExpiryDate IS NULL OR CostPrice IS NULL;
```

V3 — FK Integrity

All FK joins must return 0 orphan rows.

```
SELECT 'Medicines → Category orphans', COUNT(*)
FROM Medicines m
LEFT JOIN Category c ON m.CategoryID = c.CategoryID
WHERE c.CategoryID IS NULL;
```

V4 — Business Logic

Ensures SubTotal = QuantitySold × UnitPrice in every row.

```
SELECT 'SubTotal mismatch', COUNT(*)
FROM SaleItems
WHERE ABS(SubTotal - (QuantitySold * UnitPrice)) > 0.01;
```

V5 — Expiry Alert

Lists all in-stock batches expiring within 90 days — core business use-case.

```
SELECT m.BrandName, b.BatchNumber, b.ExpiryDate,
       DATEDIFF(b.ExpiryDate, CURDATE()) AS DaysLeft,
       b.QuantityOnHand
FROM Batches b JOIN Medicines m ON b.MedicineID = m.MedicineID
WHERE b.ExpiryDate BETWEEN CURDATE()
      AND DATE_ADD(CURDATE(), INTERVAL 90 DAY)
  AND b.QuantityOnHand > 0
ORDER BY b.ExpiryDate;
```

V6 — Staff Revenue Summary

Shows per-pharmacist sales performance — audit trail verification.

```
SELECT st.FullName, COUNT(s.SaleID) AS TotalSales,
       SUM(s.TotalAmount) AS Revenue,
       ROUND(AVG(s.TotalAmount), 2) AS AvgSale
FROM Sales s JOIN Staff st ON s.StaffID = st.StaffID
GROUP BY st.StaffID ORDER BY Revenue DESC;
```

5.4 Expected Validation Results

Validation Check	Expected Result	Status
V1 Row Counts	Staff=5, Category=8, Medicines=15, Suppliers=5, Batches=51, Sales=100, SaleItems=247	✓
V2 NULL Checks	All violations = 0	✓
V3 FK Integrity	All orphan counts = 0	✓
V4 SubTotal Logic	Mismatch count = 0	✓
V5 Expiry Alert	Varies — lists real near-expiry batches in dataset	✓
V6 Revenue Summary	3 pharmacist rows with valid sales totals	✓

GitHub Repository: github.com/awaisqarni/pharmaKeep — All SQL scripts, CSV files, and ERD assets are committed with one clear commit message per milestone (e.g., *feat(M4): add DDL script with all constraints*).